Rollout Statistics Module Audit

Murphy

2026-03-14 22:35 UTC

Scope

This audit covers the dataset-rollout statistics path that produced the five-dataset Qwen/Qwen3-1.7B bundle: `scripts/collect_model_stats.py`, the math / GPQA / MMLU-Pro / LiveCodeBench adapters under `src/loop_probe/adapters/`, `src/loop_probe/prompt_builder.py`, `src/loop_probe/collector.py`, and `slurm/run_collect_model_stats.sbatch`. I also asked Athena for an independent audit of the same module and cross-checked the returned claims against the local code and the generated summary bundle.

Executive Assessment

Athena and the local read converge on the same conclusion: the current collector is useful for descriptive loop-vs-length overlap analysis, but it is not yet a clean, benchmark-faithful measurement stack for Qwen/Qwen3-1.7B. Three issues are definite bugs or semantics bugs:

1. The math / GPQA / MMLU-Pro adapters leave Qwen3 in default thinking mode while the rollout launch stays greedy (`temperature=0.0`).
2. The reported `max_length_hit` statistic is really an "effective remaining context budget exhausted" signal, not an unambiguous "hit the model's native max length" signal.
3. Prompt-too-long samples are counted differently on QA tasks versus LiveCodeBench.

A fourth high-confidence issue is the multiple-choice grader: GPQA and MMLU-Pro score the *last* standalone answer letter anywhere in the completion, which is brittle when the model emits thought traces or extra commentary. The math verifier itself looks sound.

Current Bundle Facts

The regenerated five-dataset summary confirms that the active run used one common generation policy: Qwen/Qwen3-1.7B, `temperature=0.0`, `max_tokens=81920`, and `max_model_len=40960`. All five dataset rows in `cross_dataset_rollout_summary.json` report `prompt_too_long = 0`, so the prompt-too-long inconsistency did not fire in the shipped bundle; it is still a real latent bug for future runs.

Findings

1. **Definite bug: Qwen3 default thinking-mode prompts are paired with greedy decoding on the chat-template adapters.**

   The math, GPQA, and MMLU-Pro prompt builders all call `tokenizer.apply_chat_template(..., add_generation_prompt=True)` without explicitly setting `enable_thinking=False` (`src/loop_probe/prompt_builder.py:4-10`, `src/loop_probe/adapters/multiple_choice_gpqa.py:87-102`, `src/loop_probe/adapters/multiple_choice_mmlupro.py:94-112`).
   The rollout path then samples with `temperature=0.0` (`scripts/collect_model_stats.py:450-458`), and the Slurm wrapper keeps that as the default (`slurm/run_collect_model_stats.sbatch:94-100`). The Qwen3 model card says reasoning/thinking is enabled by default and explicitly warns against greedy decoding in thinking mode.[1]

   Why it matters: these three datasets can overstate loopiness or depress accuracy for configuration reasons rather than task reasons. This is not a cosmetic prompt issue; it changes the measurement contract itself.

   Recommended fix: either set `enable_thinking=False` for this collector family, or switch the collector to a Qwen-recommended non-greedy decoding policy and document it as intentional.

2. **Definite semantics bug: `max_length_hit` does not mean what the report language suggests.**

   The collector defines

   $$\texttt{effective_max} = \min(\texttt{max_tokens},\ \texttt{max_model_len} - \texttt{prompt_len})$$

   in `scripts/collect_model_stats.py:152-153`, and marks a hit only when `finish_reason == "length"` and `token_count == effective_max` (`scripts/collect_model_stats.py:479-490`). With the shipped defaults `max_tokens=81920` and `max_model_len=40960` (`slurm/run_collect_model_stats.sbatch:99-100`), the operative cap is usually the configured remaining context budget, not the nominal `max_tokens`. vLLM's own docs define `max_model_len` as the model context length for prompt plus output.[2]

   Why it matters: the current report wording "hit max model length" can be read as "the model hit its native context limit" even though the implementation is only detecting exhaustion of the configured local cap. This is especially risky because the Qwen3-1.7B model card advertises a 32,768-token context window, while the collector is configured to 40,960.

   Recommended fix: rename the metric to something like `effective_context_cap_hit`, log both prompt length and configured cap, and either justify the 40,960 override or reduce it to a validated value.

3. **Definite bug: prompt-too-long rows are scored inconsistently across tasks.**

   When `effective_max < 1`, QA tasks increment `num_prompt_too_long` and then skip grading entirely (`scripts/collect_model_stats.py:426-447`). Since `success_fraction = num_correct / num_graded` (`src/loop_probe/collector.py:115-116`), those

[1] https://huggingface.co/Qwen/Qwen3-1.7B
[2] https://docs.vllm.ai/en/latest/serving/engine_args.html

rows vanish from QA accuracy. LiveCodeBench does something different: it still writes an `LcbSampleRecord` with empty code and passes that record into native grading (`scripts/collect_model_stats.py:433-445`, `src/loop_probe/adapters/livecodebench_codegen.py:253-300`), which effectively counts the row as wrong.

Why it matters: the same failure mode inflates QA success rates but not LiveCodeBench success rates. Even though this branch's current bundle had zero such rows, the cross-task comparison contract is inconsistent in code.

Recommended fix: either exclude prompt-too-long rows from grading on *all* tasks, or count them as wrong on *all* tasks. The branch should pick one policy and serialize it in the report metadata.

4. **High-confidence grading bug path: GPQA and MMLU-Pro parse the last standalone option letter anywhere in the response.**

   Both graders regex the completion for option letters and score `matches[-1]` (`src/loop_probe/adapters/multiple_choice_gpqa.py:105-109`, `src/loop_probe/adapters/multiple_choice_mmlupro.py:115-135`). The prompts only ask for "the single best answer letter" and do not force a structured terminal field (`src/loop_probe/adapters/multiple_choice_gpqa.py:93-102`, `src/loop_probe/adapters/multiple_choice_mmlupro.py:103-112`).

   Why it matters: if the model emits reasoning, re-lists options, or leaves a thought trace before the final answer, the scorer can assign the wrong label even when the model "meant" the correct option. The earlier thinking-mode bug increases the chance of exactly this failure mode.

   Recommended fix: require a structured final field (for example a terminal `Final answer: X` or JSON answer slot) and parse only that field. One caveat: the current collector does not checkpoint raw MCQ completions, so this bug path is proven by the parser logic but cannot be quantified retroactively on the shipped bundle.

5. **Risk / assumption: the LiveCodeBench path is intentionally non-canonical but is easy to read as canonical.**

   The adapter maps every `qwen/*` model to `LMStyle.CodeQwenInstruct` (`src/loop_probe/adapters/livecodebench_codegen.py:181-193`), which is a reasonable guess but still a local assumption for Qwen3-1.7B. The collector also hard-codes one sample per prompt (`n=1`) and the current launch policy keeps `temperature=0.0` (`scripts/collect_model_stats.py:450-458`, `slurm/run_collect_model_stats.sbatch:94-100`). Upstream LiveCodeBench code generation instructions instead use `n=10`, `temperature=0.2`, and the parser defaults to `num_process_evaluate=12` and `timeout=6`; the local adapter defaults to `1` and `5` (`src/loop_probe/adapters/livecodebench_codegen.py:100-118`).[3] [4]

   Why it matters: the current LiveCodeBench number is perfectly usable as deterministic task-local telemetry, but it is not benchmark-canonical. The report should say so explicitly.

   Recommended fix: either align generation and evaluation with the upstream benchmark contract, or label the output as a local single-rollout deterministic variant.

[3] https://github.com/LiveCodeBench/LiveCodeBench/blob/main/README.md
[4] https://github.com/LiveCodeBench/LiveCodeBench/blob/main/lcb_runner/runner/parser.py

What Looks Correct

- **Math verification is in good shape.** The math prompt asks for a boxed final answer (`src/loop_probe/prompt_builder.py:4-10`), and `math_freeform.py` uses `math_verify` with boxed-priority extraction plus expression fallback (`src/loop_probe/adapters/math_freeform.py:24-91`). The verifier call order is also correct: `verify(parsed_gold, parsed_pred)` at `src/loop_probe/adapters/math_freeform.py:83-85`.
- **The overlap counters themselves are straightforward.** Once a completion is generated, the bookkeeping for loop counts, length sums, and overlap counts in `scripts/collect_model_stats.py:492-531` and `src/loop_probe/collector.py:114-156` is internally coherent. The dominant risks are in the prompt/decode/grading contract around those counters, not in the counter arithmetic.

Recommended Repair Order

1. Fix the Qwen3 prompt/decode contract first. This is the highest-impact issue for all chat-template datasets.
2. Rename and re-document the "max length hit" metric before anyone treats it as a literal native-model ceiling statistic.
3. Unify prompt-too-long handling across QA and LiveCodeBench.
4. Replace MCQ letter scraping with a structured terminal-answer parser and add raw completion checkpointing for auditability.
5. Decide whether LiveCodeBench is meant to be canonical benchmark evaluation or a local deterministic telemetry variant, then encode that decision in the report text and runtime defaults.

Bottom Line

The current five-dataset bundle is still informative for one descriptive claim: looped rollouts and effective context-cap terminations overlap strongly under this runtime. What it does *not* yet provide is a clean apples-to-apples statement of Qwen3-1.7B task accuracy or canonical benchmark performance, because the prompt, decode, and grading contracts are not yet consistent enough for that stronger claim.